

Data Mining 2 Project

Camilla Maffeis

Giulia Fabiani

A.Y. 2024/2025

CONTENTS

1	Data Understanding and Preparation	2
1.1	Data Semantics, Duplicates and Train/Test Split	2
1.2	First Global Exploration of the Training Set	2
1.3	Distribution of Categorical Variables and Statistics	2
1.4	Distribution of Numeric Variables and Statistics	3
1.5	Variable Transformation, Pairwise Correlations and Elimination of Variables	4
1.6	Handling Missing Values	4
1.7	Clustering	6
2	Advanced Data Pre-Processing	6
2.1	Anomaly detection	6
2.2	Imbalanced Learning	6
2.2.1	Resampling techniques	6
2.2.2	Decision threshold and assessment	7
3	Advanced Classification	8
3.1	Pre-processing	8
3.2	Logistic Regression	9
3.3	Linear Support Vector Classifier	9
3.4	Support Vector Classifier with Radial Basis Function	9
3.5	LightGBM Classifier	9
3.6	Multilayer Perceptron Classifier	9
3.7	Conclusions	9
4	Advanced Regression	9
4.1	Random Forest Regressor	9
4.2	XGBoost Regressor	9
5	Time Series Analysis	9

1 DATA UNDERSTANDING AND PREPARATION

The goal of this project is to analyze two datasets providing different kinds of information about movies, TV shows, and other forms of visual entertainment.

The tabular dataset is an extension of the one proposed in DM1: the records include key information about titles, insights into critical aspects such as awards and reviews, as well as ratings and further data about the title's production. The time series dataset, on the other hand, represents the domestic daily gross income of movies at the box office, measured from their release, complemented with information about the film's average rating and genre.

In the following section, we'll focus on the tabular dataset, while the overview about the time series dataset composition and preparation can be found in Section 5.

1.1 Data Semantics, Duplicates and Train/Test Split

The tabular dataset contains 149,531 records and 32 attributes; the nine attributes that were added to the original set from the DM1 dataset are reported in Tab. 1. This makes a total of nine categorical attributes, three binary attributes and 20 numerical attributes. Among the latter, only *runtimeMinutes* and *averageRating* can be classified as continuous, as the others mostly represent counts.

A first data quality check revealed only ten genuine duplicates, i.e. records sharing all feature values, which we dropped.

We then split the dataset, reserving 70% for training, and the remaining 30% for testing. As to avoid data leakage, we performed all subsequent data exploration exclusively on the training set. At the end of the data preparation stage, we applied all transformation and cleaning steps to the test set as well.

1.2 First Global Exploration of the Training Set

A first global analysis of the dataframe's summary and descriptive statistics revealed missing values for *endYear* (100,729), *runtimeMinutes* (28,185), *countryOfOrigin* (27,961) and *genres* (1,887). Since most records are missing an *endYear*, and the fea-

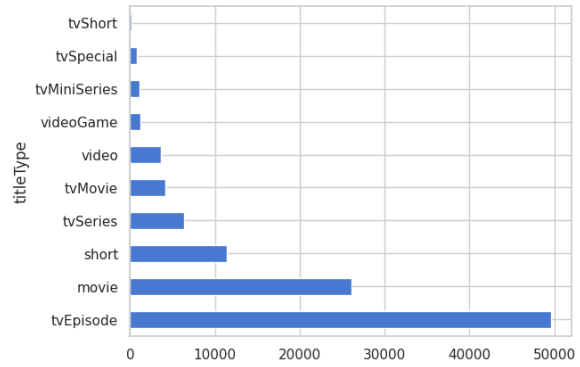


Figure 1: Distribution of *titleTypes*.

ture is strongly correlated to *startYear*, we dropped it.

Since a great majority of records had no information about sound mixes, we decided to drop this feature as well.

bestRating, *worstRating*, and *isRatable* were once again confirmed to be uninformative, as they only take one value for all records (10, 1, and 1, respectively), and were therefore dropped. We also dropped *ratingCount*, as it was strongly correlated to *numVotes*, and *isAdult*, because it's redundant w.r.t. the *adult* category in *genres*.

Checking the values for the only remaining original binary attribute, *canHaveEpisodes*, we noticed that only two records with a *titleType* different from *tvSeries* and *tvMiniSeries* have a positive value. We corrected them: the first had been registered as a *tvMovie* instead of *tvSeries*, while the second was a *short* with no episodes.

Lastly, we removed the string '\\N', which was prepended to all lists of *regions*, making the feature consistent with *numRegions*.

1.3 Distribution of Categorical Variables and Statistics

In this subsection, we studied the univariate and bivariate distribution of categorical variables, which all turned out to be unbalanced.

Differently from the DM1 dataset, the most popular *titleType* is *tvEpisode* (47% of the records), followed by *movie* (25%) and *short* (11%); *tvSpecial* (0.79%) and *tvShort* (0.17%) are the classes with the lowest support (cf Fig 1). With regard to *genres*, the two most popular classes are by far *Drama* and *Comedy*, although it depends on *titleType*. Binned *rating* follows a left-skewed distribution, with the mode being the [7,8) class (Fig. 2). The most popular *countryOfOrigin* and *region* is the US, but both features have a substantial

Feature	Description	Type
castNumber	Total Number of Cast individuals present within the IMDb title page.	Discrete. Ratio-scaled.
companiesNumber	Total Number of companies that worked for the title.	Discrete. Ratio-scaled.
regions	The regions for this version of the title. Some titles can belong to more than one class.	Categorical. 171 unique classes.
averageRating	Weighted average of all the individual user ratings.	Continuous. Ratio-scaled.
externalLinks	Total Number of External Links the title has within the IMDb page.	Discrete. Ratio-scaled.
quotesTotal	Total Number of quotes the title has within the IMDb page.	Discrete. Ratio-scaled.
writerCredits	Total number of writer credits of the title.	Discrete. Ratio-scaled.
directorCredits	Total number of director credits of the title.	Discrete. Ratio-scaled.
soundMixes	Technical specification of the sound mixes available for the title. Some titles can belong to more than one class.	Categorical. 59 unique classes.

Table 1: New features in the extended dataset.

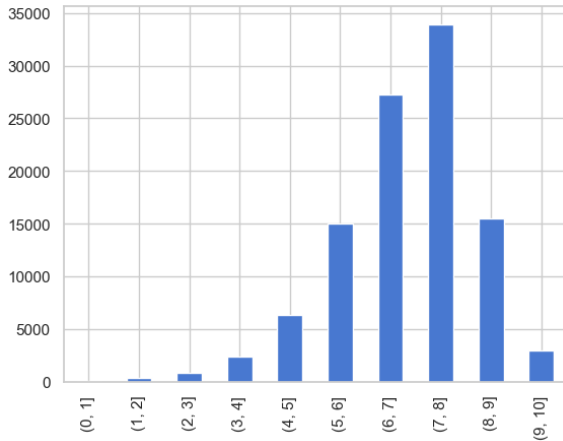


Figure 2: Barplot for rating.

amount of missing values: *nan* is the second most popular value for both.

We dropped *soundMixes*: not only are most of its 59 unique values very rare, it has too many missing values to provide useful information.

1.4 Distribution of Numeric Variables and Statistics

Before analyzing numeric variables, we added the same features we had added to the DM1 dataset:

- *numCountriesOfOrigin*: the number of countries listed for the record for the variable *countryOfOrigin*. For the training set, the values range from 1 to 10.
- *numGenres*: the number of genres listed in *genres*. For the training set, the values range from 1 to 3.
- *reviewsTotal*: the sum of *criticReviewsTotal* and *userReviewsTotal*.
- *criticReviewsRatio*: the proportion of review from critics (*criticsReviewTotal*) on the total

number of reviews of a record (*reviewsTotal*). For unreviewed records, it is set to zero.

- *awardsAndNominations*: the sum of *awardWins* and *awardNominationsExcludeWins*.

A pairplot of all numeric variables, with KDE graphs on the diagonal, allowed us to gain a global view of both the features' univariate distribution and their pairwise correlations. Given the size of the dataset, we resorted to using a random sample of 20,000 records. The great majority of the variables has a right-skewed distribution (*writerCredits*, *numRegions*, *awardWins*, *totalImages*, *criticReviewsTotal*, *runtimeMinutes*, *userReviewsTotal*, *totalCredits*, *totalVideos*, *castNumber*, *numVotes*, *quotesTotal*, *awardNominationsExcludeWins*, *companiesNumber*, *externalLinks*, *awardsAndNominations*, *numCountriesOfOrigin*, *reviewsTotal*). *averageRating*, of course, follows the same left-skewed distribution observed for its discretized version, *rating* (cf. Fig. 2).

We observed that some records have an implausibly long runtime, probably due to the inconsistent strategy adopted for the computation of the runtime for serialized content, for which sometimes the total runtime of a series is registered, rather than the average episode runtime. In order to avoid this problem, we removed the runtime of all *tvSeries*, *tvMiniSeries*, and *tvEpisodes* for which the value exceeded the 3rd quartile + 1.5 interquartile range for its specific category (105, 382.5 and 84 minutes, respectively); a total of 1560 records in the training set were affected.

A similar problem was found for *writerCredits*: records with suspiciously high values are likely TV episodes for which all writers of the entire TV series were erroneously credited. Therefore, we decided to remove the values for *writerCredits* from 2100 *tvEpisodes*, as they exceeded the 3rd quartile + 1.5 interquartile range (8.5 credits).

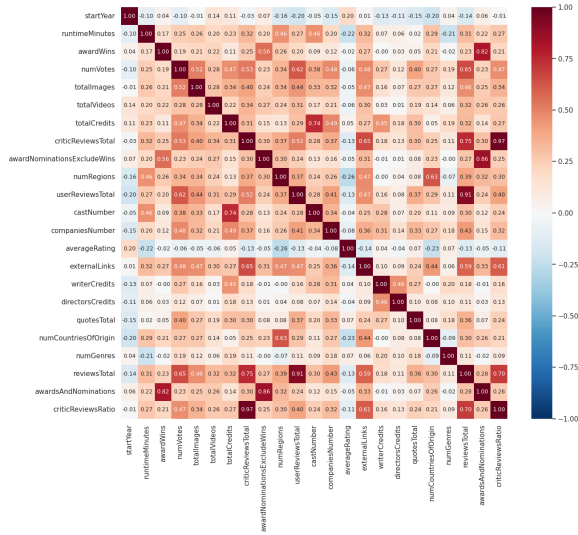


Figure 3: Correlation heatmap before attribute removal.

1.5 Variable Transformation, Pairwise Correlations and Elimination of Variables

We log-transformed all right-skewed variables, and used the Shapiro-Wilk test to determine whether they now resembled a normal distribution. None of them did, therefore we used Spearman’s correlation coefficient instead of Pearson in plotting the correlation heatmap in Fig 3.

We removed the following features because they were highly correlated to another feature: *awardNominationsExcludeWins* (0.86, correlated with *awardsAndNominations*), *castNumber* (0.74, with *totalCredits*). Since all features regarding reviews and *numVotes* were highly correlated with each other, we dropped them all with the exception of *numVotes*, which has the least skewed distribution, and *criticReviewsRatio*, which offer insights of a different nature. We however decided to conserve information about the presence/absence of reviews by users and critics with the creation of two new boolean features: *hasUserReviews* and *hasCriticsReviews*.

Moreover, we binarized features which overwhelmingly took on the value 0 (or 1, in the case of *numCountriesOfOrigin* and *directorsCredits*). The results of these transformations are reported in Table 2.

The total number of final features is 27; they are listed in Tables 3 and 4. Among the remaining numeric features, the heatmap in Fig. 4 shows that *externalLinks* exhibits a moderate to high positive correlation with *criticReviewsRatio* (0.61) and a low to moderate correlation with *numRegions*, *numVotes* and *totalImages* (around 0.47), while

Original feature	Boolean version	% True
awardWins	hasAwards	8.16
awardsAndNominations	hasAwardsOrNominations	12.25
totalVideos	hasVideos	7.93
criticReviewsTotal	hasCriticsReviews	24.88
userReviewsTotal	hasUserReviews	36.98
directorsCredits	moreDirectorsCredits	9.89
quotesTotal	hasQuotes	13.86
numCountriesOfOrigin	moreCountriesOfOrigin	5.05

Table 2: Binarization of numeric features.

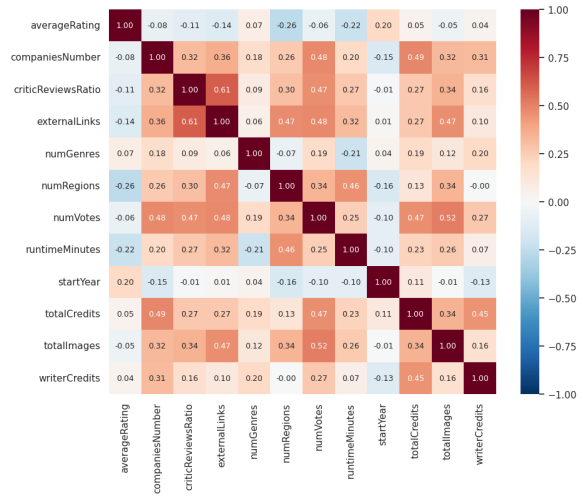


Figure 4: Correlation heatmap after attribute removal.

numVotes is also mildly correlated to *totalImages*, *totalCredits*, *criticReviewsRatio*, and *companiesNumber* (between 0.47 and 0.52).

1.6 Handling Missing Values

At this point, we still have missing values for *countryOfOrigin*, *region*, *genres*, *runtimeMinutes* and *writerCredits*; some were preexisting, others were removed as detailed Subsec. 1.4. Since the distribution of *runtimeMinutes* depends on *titleType*, and the same holds true for *writerCredits*, we imputed their missing values using the median value within each respective class.

On the other hand, we chose not to replace missing values for *genres*, *region* or *countryOfOrigin*. They can easily be handled when performing the encoding for our methods without introducing bias, therefore we’d rather preserve the original distributions and correlations.

Feature	Description	Type
runtimeMinutes	Primary runtime of the title, in minutes.	Continuous. Ratio-scaled.
startYear	Represents the release year of a title. In the case of TV Series, it is the series' start year.	Discrete. Interval.
numVotes	Number of votes the title has received.	Discrete. Ratio-scaled. Log-transformed.
numRegions	The regions number for this version of the title.	Discrete. Ratio-scaled. Log-transformed.
totalImages	Total number of images for the title within the IMDb title page.	Discrete. Ratio-scaled. Log-transformed.
totalCredits	Total number of credits for the title.	Discrete. Ratio-scaled. Log-transformed.
criticReviewsRatio	The proportion of reviews from critics on the total number of reviews of a record. For unreviewed records, it is set to 0.	Continuous. Ratio-scaled.
numGenres	The number of genres listed for the variable <i>genres</i> .	Discrete. Ratio-scaled.
companiesNumber	Total number of companies that worked for the title.	Discrete. Ratio-scaled.
externalLinks	Total number of external links the title has within the IMDb page.	Discrete. Ratio-scaled.
writerCredits	Total number of writer credits of the title.	Discrete. Ratio-scaled.
averageRating	Weighted average of all the individual user ratings.	Continuous. Ratio-scaled.

Table 3: Numeric attributes after data preparation.

Feature	Description	Type
originalTitle	Original title, in the original language.	Categorical; mostly unique classes (i.e. names).
titleType	The type/format of the title (e.g. movie, short, tvseries, tvepisode, video, etc.).	Categorical, 10 classes.
countryOfOrigin	The country where the title was primarily produced.	Categorical, 153 classes. Some titles can belong to more than 1 class.
genres	The genre(s) associated with the title (e.g., drama, comedy, action).	Categorical, 29 classes. Some titles can belong to more than 1 class.
regions	The regions for this version of the title.	Categorical, 171 unique classes. Some titles can belong to more than 1 class.
rating	IMDb title rating class.	Ordinal, 10 values.
hasAwards	Whether the title has won at least an award	Asymmetric binary.
hasAwardsOrNominations	Whether the title was at least nominated for an award.	Asymmetric binary.
canHaveEpisodes	Whether the title can have episodes.	Asymmetric binary.
hasVideos	Whether the title IMDb title page has at least one video.	Asymmetric binary.
moreCountriesOfOrigin	Whether the title was produced in more than one country.	Asymmetric binary.
hasCriticsReviews	Whether the title has any critic reviews.	Asymmetric binary.
hasUserReviews	Whether the title has any user reviews.	Asymmetric binary.
moreDirectorsCredits	Whether the title has more than one director credits.	Asymmetric binary.
hasQuotes	Whether the title has any quotes within the IMDb page.	Asymmetric binary.

Table 4: Categorical, ordinal, and binary attributes after data preparation.

1.7 Clustering

As part of the data understanding phase, we conducted hierarchical clustering on the dataset, using all numeric attributes.

Since the size of the dataset made it unfeasible to perform hierarchical clustering directly because of the computational cost of computing distances between all records, we firstly reduced our dataset to a more manageable 10,000 records, running k-means clustering with $k=10,000$.

We then performed hierarchical clustering on the centroids of K-Means clusters, experimenting with different linkage methods and inspecting the resulting dendrograms. We found that using average linkage yielded the most satisfactory results. We then cut the dendrogram at an appropriate threshold and, after propagating cluster labels back to the entire dataset, analyzed the composition of the clusters.

We found 13 clusters of varying sizes, with the top cluster grouping one fifth of the entire dataset and the top four clusters grouping half of the dataset. Five clusters contained less than 5,000 records each.

For reasons that will become apparent in 2.1, we concentrated our analysis on two smaller clusters: C1 (1,668 records) and C2 (3,923). These clusters belong to the same supercluster, that is merged latest of all with the rest of the dataset.

Both clusters are made up primarily from movies (with some tv series), and according to their distribution w.r.t. *numVotes*, probably contain blockbuster movies and very popular TV series 5. In addition to that, almost 57% of records from C1 received awards, a proportion that increases to 80% when considering also nominations. C2 records were also well received, with 45% of them having received or being considered for awards. In addition to that, 87% of records from C1 and 45% of records from C2 have videos on their IMDB page and almost all records from both C1 and C2 have both reviews from users and from critics.

2 ADVANCED DATA PRE-PROCESSING

2.1 Anomaly detection

2.2 Imbalanced Learning

In this section we will look at several imbalance learning techniques in the context of binary classification.

Resampling Method	precision	recall	f1
No strategy	.47 ± .02	.31 ± .01	.37 ± .01
Random OS	.29 ± .00	.62 ± .01	.40 ± .01
SMOTE	.29 ± .00	.65 ± .01	.40 ± .01
ADASYN	.30 ± .00	.59 ± .01	.40 ± .00
Random US	.24 ± .00	.86 ± .01	.37 ± .00
Edited NN	.50 ± .01	.45 ± .01	.47 ± .01
SMOTE + Edited NN	.30 ± .01	.68 ± .01	.41 ± .01

Table 5: Results of tested resampling methods on 5-fold cross validation.

Our goal was to develop a system that is capable of predicting which titles have received any awards (as already seen in 2, only about 8% of our training records did). We figured that a similar system could be useful in a number of contexts, like for recommendation and showcasing purposes or for streaming platforms attempting to decide which titles to license when this information isn't already available (for example for newly released titles).

2.2.1 Resampling techniques

We established a performance baseline using a KNN classifier with no resampling techniques. We then experimented with three over-sampling strategies (Random Over-sampling, SMOTE, ADASYN), two under-sampling strategies (Random Under-sampling and Edited Nearest Neighbors) and a combination of both (SMOTEEN). In all cases we resampled the dataset as to obtain a perfect balance between the positive and the negative class.

For each technique we fine tuned the hyperparameters of the classifier searching for the best F1 score: initially we performed random search on k s between 1 and 100, with euclidean and manhattan distance, both weighting neighbors and not. Since we noticed generally better performance with weighting of exemplars, we removed unweighted models from the grid search.

We also noticed that for over-sampling methods performance was rapidly decreasing as k got bigger, so we restricted our grid search to odd k s between 1 and 21. Since performance of under-sampling techniques wasn't degrading so rapidly, we extended the number of neighbors considered for these approaches to odd numbers between 1 and 41, but we noticed that scores were nevertheless plateauing around 10, so we didn't consider it necessary to widen the search beyond that.

We then compared performance of different resampling strategies checking precision, recall and F1 score on 5-fold cross validation (cf 5).



Figure 5: Hierarchical clustering results on tabular data by runtime and number of votes.

Precious insights were also gathered by the inspection of the precision-recall curves (reported in fig 6) and average precision.

Over-sampling methods and the combination of SMOTE and Edited NN all show worse ranking capabilities than no resampling strategy at all. In particular these methods demonstrate much worse performance in the early retrieval range of operating points, meaning that they have much more false positives in the top ranked instances. This is also reflected in the lower precision scores that are not balanced by the higher recall. In addition to that, over-sampling techniques, even when relatively inexpensive at fitting time (Random Over-sampling), are slower at prediction time (because of the necessity to compute distances with all records in the over-sampled dataset) and have considerably higher space requirements.

Performance obtained with Random Under-sampling has similar F1 score to no resampling strategy, but shows better ranking capabilities, as demonstrated both by AP score and inspection of the PR curve.

The best results are obtained with Edited Nearest Neighbor. The PR curve for ENN almost always dominates random under-sampling with minimal crossing-over for higher values of recall.

It seems to us that Random Under-sampling may be the preferred choice in some context in which fitting time would be a more pressing concern than the precision of the prediction, so we

decided to assess both Edited NN and Random Under-sampling on the test set.

2.2.2 Decision threshold and assessment

Edited Nearest Neighbor showed satisfactory performance, improving by 0.1 the F1 score of the vanilla KNN and having the best precision among all tested methods (0.50) and recall at least better than the vanilla KNN. Nevertheless, we decided to post-tune the decision threshold of both our best performing models before assessment on the test set. For the pipeline using Random Under-sampling this resulted in setting a higher threshold (0.77), while for Edited NN the threshold had to be lowered to 0.38. Fig 7 illustrates the effect of changing the decision threshold for the pipeline using Random Under-sampling: we can observe that at the default threshold the model would have better recall but much worse precision than at the tuned threshold.

We finally assessed both models on the test set, obtaining very similar results: ENN obtained a F1 score of 0.49, while Random Under-sampling 0.47; inspecting the other metrics we can confirm that the worse performance of Random Under-sampling is entirely due to a lower precision (0.42 vs 0.44 for ENN), as we had expected from the inspection of the PR curves in validation.

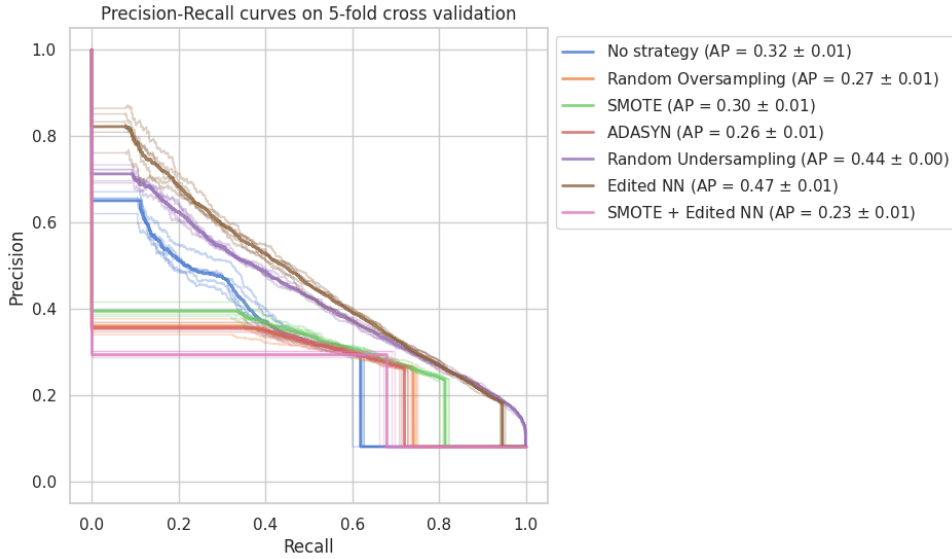


Figure 6: Precision-Recall curves on 5-fold cross-validation for different resampling methods

3 ADVANCED CLASSIFICATION

In this section, we will examine the performance of various advanced classification methods w.r.t two multiclass classification tasks, aiming to predict the *rating* class and the *titleType* of our records.

Since no features showed high correlation with *averageRating*, while some did exhibit different distribution tendencies w.r.t. *titleType*, we anticipated that the second classification task would yield better results.

In evaluating and comparing our classification models, we selected macro F1 as the main performance metric, given that the different target classes have variable support. For each model, we additionally plotted the Precision-Recall (PR) curves for each class using the one-vs-rest strategy and reported its PR AUC. We favored PR curves instead of ROC curves because they are more informative in multiclass and imbalanced settings.

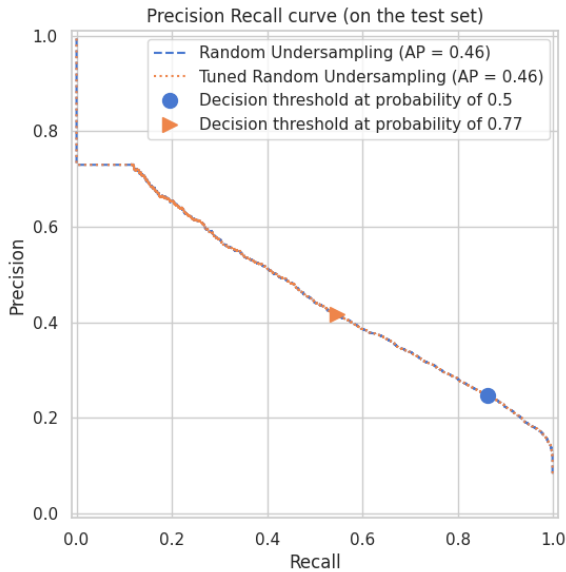


Figure 7: Decision threshold for KNN with Random Under-sampling before and after tuning.

3.1 Pre-processing

For each model, we built a pipeline which proceeded to first preprocess the data (we used the training and test set, cleaned and prepared as detailed in Sec. 1) and then pass it to the model, in order for it to be fitted on the training set or applied on the test set.

The preprocessing step included scaling the numerical features, if required by the method, and one-hot encoding the categorical variables.

For both tasks, we of course dropped the target variable, including *averageRating* for the rating task.

For *countryOfOrigin* and *region*, we only kept the 25 most frequent countries, aggregating the rest into an *Other* category, to reduce dimensionality. For the MultiLayerPerceptron Classifier (and, later, for the regression task), we further reduced dimensionality, keeping only the top nine countries for both features and dropping *hasAwards* on the account of its positive correlation to *hasAwardsOrNominations*, in order to further reduce computational costs and training time during hyperparameter tuning.

3.2 Logistic Regression

3.3 Linear Support Vector Classifier

3.4 Support Vector Classifier with Radial Basis Function

3.5 LightGBM Classifier

3.6 Multilayer Perceptron Classifier

3.7 Conclusions

4 ADVANCED REGRESSION

In this section, we will discuss the performance of two advanced regression methods in predicting the *averageRating*.

We used the same preprocessing pipeline detailed in Subsec. 3.1 and the feature set we had used for the Multilayer Perceptron.

We evaluated performances using the Determination Coefficient (R^2) and the Mean Squared Error (MSE). To set a baseline, we used a DummyRegressor which always predicts the mean of the target values observed in the training set, which yielded a R^2 of 0 and a MSE of 1.819 (equal to the variance of the target variable).

4.1 Random Forest Regressor

4.2 XGBoost Regressor

5 TIME SERIES ANALYSIS